

GPUリソース配分 アーキテクチャ検討レポート

vLLMを用いた推論基盤において、サービスごとにGPUリソースを柔軟に割り当てる構成は成立するのか。単一ノードから複数ノードまでの選択肢を整理し、配分方式の選定基準と、費用をかけずに始められる検証手順をまとめたものである。

作成日 2026-07-23 版 第3版 対象 vLLM v0.25.1 / Kubernetes v1.36 想定読者 GPU基盤の設計・検証担当

目次

00 このレポートについて

01 用語集

02 3つの層の関係

03 ハードウェアの基礎

3.1 GPU・サーバ・基板

3.2 PCIe / NVLink / NVSwitch

3.3 SXM と NVL

04 並列化の基礎

05 vLLMのメモリの仕組み

06 配分方式の比較

07 構成パターン集

08 k8s か ベアメタルか

09 ベアメタル構成の実装

9.2 直接記載の禁止

9.4 ゲートウェイの実装

9.5 ゲートウェイのリスク

10 検証計画

11 調達の指針

12 ケーススタディ

A コマンド早見表

B 除外技術（MIG）

C 出典一覧

このレポートについて

本レポートは、GPUサーバ上で複数のLLM推論サービスを動かす際の「リソース配分」に絞って整理したものである。

扱う問い

- 単一または複数のGPUノードで、サービスごとにGPUメモリを柔軟に割り当てられるか
- 1枚のGPUを複数サービスで分け合うべきか、それとも丸ごと割り当てるべきか
- Kubernetesは必須か。ベアメタル構成でも同じ制御ができるか
- 本番投入前に、どこまでを無償で検証できるか

検討対象から除外した技術

GPUをハードウェアレベルで分割する **MIG** は、検討の結果**本構成では採用しない**。判断の根拠は付録Bに記載する。本文はMIGを使わない前提で構成している。

情報の鮮度について

確認済みの前提

- vLLM の最新版は **v0.25.1**（2026年7月14日 PyPI 公開）である
- Kubernetes の最新版は **v1.36「Haru」**（2026年4月22日リリース）である
- DRA（後述）は Kubernetes 1.34 で正式版となり、1.35 で機能ゲートが固定された
- **LiteLLM** のクリーンな最新系列は **v1.83.0 以降**である（第9章の事案を参照）

推定値の扱い

第11章のKimi K3に関する必要GPU枚数は、公開されているパラメータ数からの**概算**である。同モデルの重みは本レポート作成時点で未公開であり、実測に基づく数値ではない。

用語集

本レポートで繰り返し登場する語を先にまとめておく。以降の章では、この定義を前提に記述する。

用語	意味
VRAM	GPUに搭載されたメモリ。GPUメモリ、HBMとも呼ぶ。CPUが使うRAMとは別物である。
HBM3e	H200などが採用する高帯域メモリの規格。VRAMの一種と考えてよい。
KVキャッシュ	生成中の会話の途中経過を保持する作業領域。同時に処理できるリクエスト数を決める。
TP	テンソル並列。各層の計算を横に分割し、複数GPUで同時に処理する方式。
PP	パイプライン並列。モデルを層で輪切りにし、GPU間でバケツリレーする方式。
DP	データ並列。モデルの完全な複製を各GPUに置き、リクエストを振り分ける方式。
MPS	Multi-Process Service。複数プロセスが1枚のGPUを効率よく共有するための仕組み。
NVLink	GPU同士を直結する専用の高速配線。PCIeより桁違いに速い。
NVSwitch	NVLink用の交換機。8枚のGPUを相互に全結合するために使う。
HGX	GPUとNVSwitchを載せた基板の名称。サーバそのものではない。
DRA	Dynamic Resource Allocation。Kubernetesがハードウェアを条件で要求できる仕組み。
MoE	Mixture of Experts。多数の「専門家」から一部だけを使う省計算アーキテクチャ。
ゲートウェイ	複数の推論エンジンの手前に置く中継役。クライアントには1つの窓口に見せ、内部で振り分ける。
OpenAI互換API	OpenAIが定めたリクエスト形式に準拠したAPI。クライアントは接続先を意識せずに済む。
LoRA	ベースモデルに小さな差分だけを足して用途別に振る舞いを変える手法。アダプタはMB単位。
LWS	LeaderWorkerSet。複数のPodを1セットとして扱うためのKubernetesの拡張。
FP8 / FP4	数値の精度。ビット数が少ないほどメモリを食わないが、対応するGPU世代が限られる。

3つの層の関係

vLLM・GPUノード・Kubernetesの3つは、しばしば同じ文脈で語られるが、それぞれ異なる層に属している。本章ではその関係を整理する。

最初に押さえるべき点

必須なのは **GPUノード** と **vLLM** の2つだけである。**Kubernetes**は任意の追加レイヤであり、無くても推論基盤は成立する。



2.1 決めていることは重ならない

3層がきれいに分業しているのが、この構成の理解しやすいところである。

層	決めること	決められないこと
GPUノード	何が何枚あるか、NVLinkの有無、GPU同士がどうつながっているか	誰が使うか
Kubernetes	誰にどれを渡すか、記帳と再配置	渡した中身の使われ方
vLLM	もらったGPUの中での並列化とメモリ配分	何枚もらえるか

vLLMは「GPUを2枚もらった」ことしか知らない。それが8枚搭載サーバの3番目と4番目なのか、別のノードなのかは関知しない。

逆にKubernetesは、渡したGPUの中でvLLMが `--gpu-memory-utilization` をいくつに設定しているかを知らない。

2.2 単位の違いが混乱の元になる

層	単位	性質
GPUノード	台・枚	物理的に固定されており、増減しない
vLLM	インスタンス (=1モデル)	TP × PP 枚のGPUを占有する
Kubernetes	Pod / ノード	Podはノードをまたげない

最後の1行が実務に効いてくる。

Podが1台のノードに収まる決まりであるため、**1つのvLLMインスタンスが2ノードにまたがる場合はPodが2つ必要になる**。ところが通常のDeploymentでは「この2つのPodで1セット」を表現できない。

LEADERWORKERSET が必要になる理由

これはハードウェアの都合ではなく、**Kubernetesのデータモデルの都合**である。7.3節で扱うLWSは、この表現力の穴を埋めるために存在する。

2.3 vLLMとKubernetesの接点は驚くほど細い

実際にやり取りされているのは、実質2つだけである。

```
# ① 何枚渡すか (Pod定義に書く)
resources:
  limits:
    nvidia.com/gpu: 2

# ② どれを渡したか (device plugin が自動でセットする環境変数)
CUDA_VISIBLE_DEVICES=3,4
```

vLLMは自分がKubernetes上で動いていることを**認識していない**。ただ「見えるGPUが2枚ある」と思って起動しているだけである。

この事実の帰結

だからこそ、第8章で詳述するとおり**ベアメタルでも同一の制御が成立する**。`CUDA_VISIBLE_DEVICES` を人が書くか、device pluginに書かせるかの違いしかない。

2.4 まとめ

層	例えると	無いとどうなるか
GPUノード	器	そもそも動かない
vLLM	中身	そもそも動かない

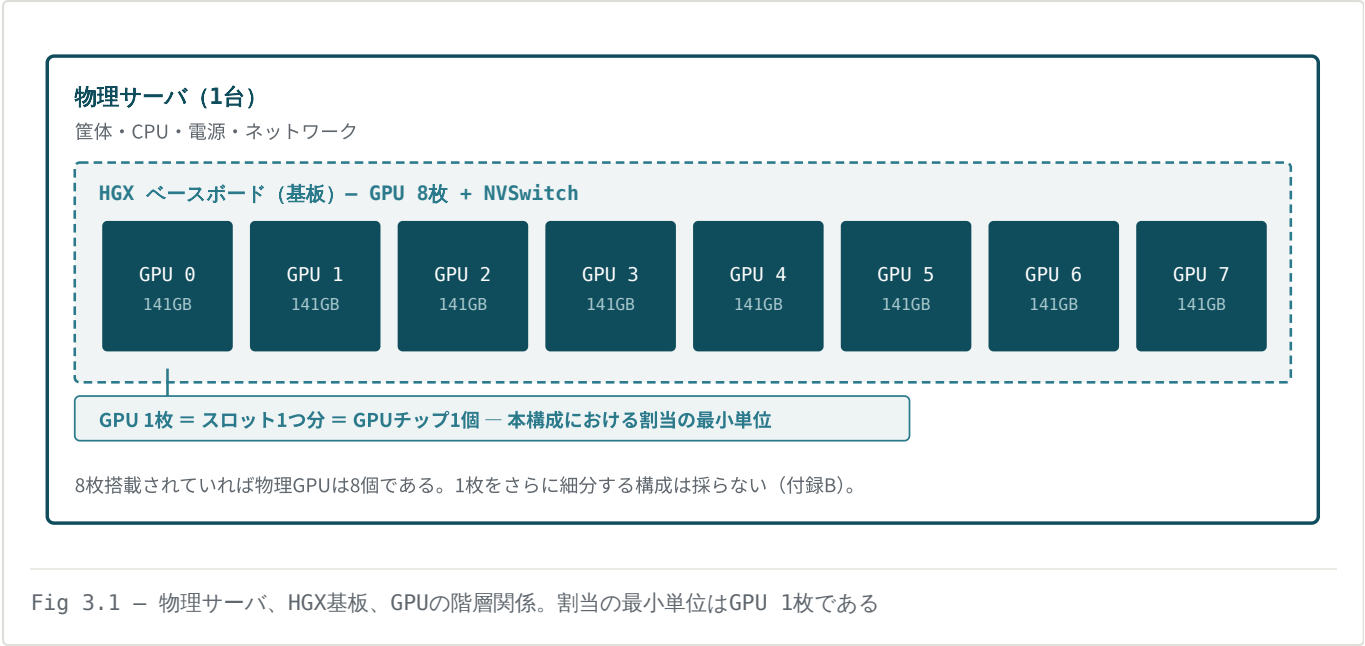
層	例えると	無いとどうなるか
Kubernetes	配車係	動く。ただし台数が増えると管理しきれなくなる

配車係がいなくても車は走る。台数が増えて、誰がどこに停まっているかを人手で把握しきれなくなったときに、初めて必要になる。

ハードウェアの基礎

3.1 GPU・サーバ・基板の関係

用語が階層になっており、混同しやすい。まず全体像を整理する。



チップとGPUの関係：「GPU 1枚」「GPUカード1枚」「GPUチップ1個」はすべて同じものを指す。本レポートでは「GPU 1枚」で統一する。

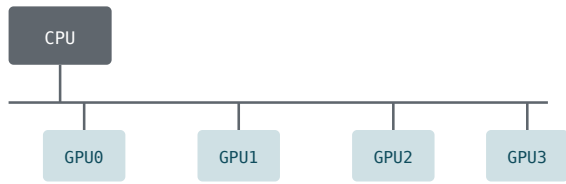
3.2 PCIe / NVLink / NVSwitch

GPU同士がデータをやり取りする経路には種類がある。道路に例えると理解しやすい。

名称	つながる相手	帯域	例えると
PCIe Gen5	CPU ⇄ GPU	約 128 GB/s	一般道。すべてのGPUが必ず持つ
NVLink	GPU ⇄ GPU のみ	約 900 GB/s	専用高速道路
NVSwitch	NVLink同士	—	高速道路のジャンクション

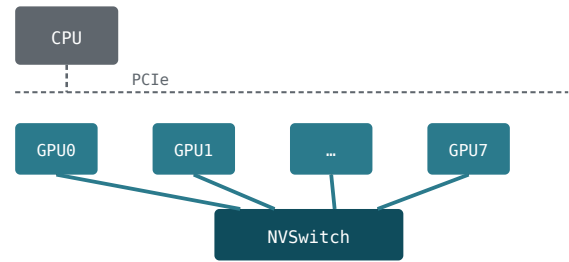
H100およびH200のNVLinkは1枚あたり最大900GB/sであり、PCIe Gen5の7倍以上の帯域を持つ。世代別ではA100 が600GB/s、H100/H200 が900GB/s、B200/GB200 が1.8TB/s である。

A. PCIe のみ (L40S など)



GPU同士の通信もPCIe経由。低速でCPUを経由する場合もある。

B. NVSwitch あり (HGX H200)



8枚すべてが900GB/sで全対全に接続される。

PCIe (点線) も同時に存在し、CPUとの通信に使う。

Fig 3.2 - PCIeのみの構成とNVSwitch構成の違い。NVLinkはPCIeを置き換えるのではなく、併存する

重要

NVLinkはPCIeの上位互換ではない。 NVLinkはGPU同士しかつながらず、CPU・ネットワークカード・ストレージとは通信できない。したがってNVLinkがあってもPCIeは必ず必要である。SXMモジュールのソケットは、ホストCPUへのPCIeと、同一基板上的他GPUへのNVLinkレーンの両方を通してている。

運用上の確認事項

- **Fabric Manager が起動していること。** NVSwitch搭載機では `nvidia-fabricmanager` がNVLinkを初期化する。停止していると、NVSwitchがあってもNVLinkが機能しない。

```
# 接続状態の確認 (NV18 等と出ればNVLink接続)
nvidia-smi topo -m
```

3.3 HGX・DGX・SXM・NVLの違い

調達時に最も誤解が生じやすい部分である。

HGXとは基板の名称である。 4枚または8枚のGPU、NVLinkインターコネクト、電源供給、熱インターフェースを含む標準化されたベースボード設計であり、サーバOEMが自社の筐体に組み込むものである。HGX単体では購入できない。

購入形態	作る主体	特徴
DGX H200	NVIDIA	CPU・メモリ・NW・ストレージ・ソフトウェア・サポートが固定セット。高価だが一体で完結する
OEMサーバ	Dell / Supermicro / HPE / Lenovo など	中身のHGX基板はDGXと同一。 最も一般的な調達形態である
H200 NVLカード	—	PCIeカード単体。汎用サーバに挿す

OEMはHGXの周囲にサーバを組むが、ベースボード自体は変更できない。GPU数、メモリ構成、NVLink配線、NVSwitchレイアウトはすべてNVIDIAが固定している。つまりトポロジは選択肢がなく、迷う必要もない。

調達仕様での必須確認事項

H200には **SXM** と **NVL** の2形態があり、NVLinkの構成が異なる。

形態	GPU間接続	冷却
H200 SXM HGX / DGX	NVSwitchで8枚を全結合。各GPU 900GB/s	液冷が基本
H200 NVL PCIeカード	2〜4枚のブリッジのみ。NVSwitchファブリックなし。TDPは600Wに低下	空冷可

仕様書には「H200 8基」ではなく **「HGX H200 (SXM5) 8基、NVSwitch搭載」** まで明記すべきである。

並列化の基礎 — TP・PP・DP

モデルを複数GPUで動かす方法は3種類ある。TPとPPはモデルを分割する方式であり、DPはモデルを複製する方式であるという点が最初の分岐になる。

TP（テンソル並列）— 層の「中」を横に切る



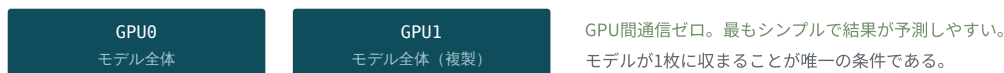
1リクエストの処理そのものが速くなる。レイテンシを下げられる唯一の方式である。

PP（パイプライン並列）— 層と層の「間」で縦に切る



枚数で割り切れなくてもよい。ただしリクエストが少ないと待機するGPUが出る（パイプラインバブル）。

DP（データ並列）— モデルを丸ごと複製する



GPU間通信ゼロ。最もシンプルで結果が予測しやすい。
モデルが1枚に収まることが唯一の条件である。

スループットを上げたい場合の第一選択となる。

Fig 4.1 — TP・PP・DPの切り方の違い。TPは層の中、PPは層の間、DPは複製である

方式	目的	通信量	必要な条件
DP	スループット向上	ゼロ	モデルが1枚に収まること
TP	レイテンシ短縮・大型モデル対応	大	NVLink推奨。attention head数で割り切れること
PP	大型モデル対応	小	特になし。不均等な分割も可能

TPの制約

TPはattention head数によって制約され、割り切れる必要がある。主要モデルのhead数は32 / 40 / 64 / 96 / 128 のように3で割り切れないものが大半であるため、**TP=3 は多くのモデルで通らない**。

GPU数で割り切れない場合、vLLM公式は**パイプライン並列の使用を推奨**している。レイヤ方向に分割するため不均等な分割に対応でき、この場合は `tensor_parallel_size=1`、`pipeline_parallel_size` をGPU数に設定する。

TPとDPの使い分け

- モデルが1枚に**余裕**で収まる → **DP（レプリカ）**。通信ゼロが効く
- モデルが1枚に**ギリギリ**収まる → **TP**。KVキャッシュの余裕が大きく増える（5.2節参照）
- レイテンシが最重要 → **TP**

- NVLinkがない → **PP** または **DP**

実測では、同時リクエスト128以下ではTP=8構成が40～86%高いスループットを示し、512以上ではDP構成が16～47%上回る。交差点はおおむね256～512同時リクエスト付近である。

vLLMのメモリの仕組み

5.1 GPUメモリの内訳

vLLMは起動時にGPUメモリを先取りして確保する。その内訳は大きく3つに分かれる。

H200 1枚 (141GB) / 70Bモデル FP8 の例

モデル重み 70GB	固定 10GB	KVキャッシュ 61GB
---------------	------------	--------------

■ 重み — 固定。動かない ■ オーバーヘッド — ほぼ固定 ■ KVキャッシュ — ここが同時リクエスト数を決める

KVキャッシュとは：生成中の会話の途中経過を保存しておく作業領域である。リクエスト1本ごとに、その文脈の長さに比例して消費する。

`--gpu-memory-utilization` は「GPU全体の何割を使うか」を指定するオプションである。複数のvLLMを1枚に同居させる場合、この割合の合計が1.0を超えないようにする。

数値は環境をまたいで移せない

中央の「オーバーヘッド」はGPU容量に比例せず、ほぼ絶対値で決まる。したがって、

- L4 (24GB) で `0.85` が安全 → H200 (141GB) では過剰に保守的になる
- H200で詰めた `0.95` を L4 に持ち込む → 即座にOOMとなる

転用できるのは「余白を何GB残すか」という絶対値の考え方と測定手順であって、パーセンテージそのものではない。

5.2 KVキャッシュと同時リクエスト数

KVキャッシュの容量が同時処理量を決める。容量が大きいほど大きなバッチと長い文脈を扱えて、どちらもスループットを上げる。実測では、KV予算を20GiBから60GiBに広げるとバッチサイズ128でスループットが58.8%向上した。

TPを増やすとKVキャッシュも増える

直感に反する点だが、重要である。TP=2にすると重みが半分ずつになるため、各GPUのKVキャッシュ用の余りが増える。

TP=1 (1枚)

重み 70GB	10	KV 61GB
---------	----	---------

TP=2 (2枚合計) - KVは約3倍に増える

35GB	10	KV 96GB	35GB	10	KV 96GB
------	----	---------	------	----	---------

合計 KV = 192GB

vLLM公式も、KVキャッシュ不足によるプリエンプションが頻発する場合の対処として `tensor_parallel_size` を増やすことを挙げている。重みをGPU間で分割するため、各GPUがKVキャッシュに使えるメモリが増えるからである。

5.3 起動ログの読み方

vLLMは起動時に、そのサービスが何リクエスト同時に捌けるかを出力する。

```
# vLLM 起動ログの該当行
GPU KV cache size: 643,232 tokens
Maximum concurrency for 40,960 tokens per request: 15.70x
```

- **GPU KV cache size** … KVキャッシュに一度に格納できるトークン総数
- **Maximum concurrency** … 1リクエストが指定トークン数を要する場合に、何本同時に捌けるかの推定値

同時リクエスト数の解釈

「同時リクエスト128」は**ユーザ数ではなく、その瞬間に生成処理中のリクエスト本数**である。ユーザが回答を読んでいる間は枠を占有しない。

1リクエストの生成が10秒、ユーザが1分に1回質問するとすれば、128枠で実効的には約768人に相当する。また128を超えてもエラーにはならず、キューで待機する。KVキャッシュ領域が不足するとシーケンスグループがプリエンプションされ再計算モードで処理されるが、これはレイテンシに悪影響を与える。**128は上限ではなく、遅くなり始める点である。**

配分方式の比較

GPUリソースを分ける方式は4つある。それぞれが実際に何を保証するかを整理する。

方式	メモリ量の強制	アドレス空間分離	障害分離	稼働中の再配分
GPU丸ごと割当 1サービス = 1枚以上	完全	あり	あり	可 (Pod入替)
vLLM複数プロセス utilization で分割	先着確保	あり	なし	sleep modeで一部可
MPS	APIレベル	あり (Volta以降)	なし	不可
Time-slicing	なし	なし	なし	不可

各方式の補足

MPS (Multi-Process Service)

MPSは `CUDA_MPS_PINNED_DEVICE_MEM_LIMIT` によりクライアントごとのメモリ上限を設定できる。ただしこれはハードウェアレベルではなくCUDA APIレベルでの強制である。Volta世代以降では各MPSクライアントが独立したGPUアドレス空間を持ち、クライアント同士がメモリを誤って読むことは防がれる。

弱点は障害分離である。1つのクライアントの致命的な障害でMPSサーバがFAULT状態に入り、同じGPUを共有する他のクライアントに影響する。NVIDIAはdevice pluginでのMPSサポートを実験的と位置づけている。

Time-slicing

すべての時分割クライアントが同じ物理VRAMプールを境界なしで共有するため、過大なメモリを確保したプロセスが同居ワークロードにOOMを引き起こす。これは設定で回避できるものではなく、アーキテクチャ上の性質である。開発環境以外での利用は推奨できない。

本構成での方針

GPU丸ごと割当を第一候補とする

H200は141GB、B200は180GBある。分割しないという選択肢を先に検討すべきである。

「GPU 1枚 = サービス1つ」は、物理的に別チップであるためメモリも障害も完全に分離される。NVLinkも無効化されず、Kubernetesから見れば `nvidia.com/gpu: 1` の1行で済む。再配分もPodの入れ替えのみで完了する。

1枚に収まらない、あるいはサービス数がGPU枚数を上回る場合に限り、上表の他の方式を検討する。具体的な適用先は第7章で扱う。

分離と柔軟性の関係

「異なる信頼境界を1枚のGPUに同居させたい」と「柔軟に配分したい」は**同時には満たせない**。強い分離を与える方式は柔軟性を奪い、柔軟な方式は分離を与えないためである。したがって本構成では、**分離はGPU / ノードの境界で取り、その内側で柔軟に配分する**という二層構造を採る。

構成パターン集

7.1 単一ノード・複数サービス

H200が8枚搭載されたサーバに、4つのサービスを配置する例を示す。これは「GPU共有」ではなく単なる**GPU割当**であり、GPUを分け合う仕組みは一切不要である。

H200 × 8枚 の割当例

サービスA 2枚 TP=2	サービスB 3枚 PP=3	サービスC 2枚 TP=2	D 1枚
GPU 0-1	GPU 2-4	GPU 5-6	GPU 7

ベアメタル / Docker の場合

```
# CUDA_VISIBLE_DEVICES = そのプロセスに見せるGPUを限定する環境変数
CUDA_VISIBLE_DEVICES=0,1 vllm serve model-a --tensor-parallel-size 2 --port 8001
CUDA_VISIBLE_DEVICES=2,3,4 vllm serve model-b --pipeline-parallel-size 3 --port 8002
CUDA_VISIBLE_DEVICES=5,6 vllm serve model-c --tensor-parallel-size 2 --port 8003
CUDA_VISIBLE_DEVICES=7 vllm serve model-d --port 8004
```

Kubernetes の場合

```
resources:
  limits:
    nvidia.com/gpu: 2 # device plugin がGPUを丸ごと排他的に割り当てる
```

3枚構成の落とし穴

サービスBのように3枚を割り当てる場合、TP=3は多くのモデルで通らない（第4章参照）。`--tensor-parallel-size 1 --pipeline-parallel-size 3` とするか、レプリカ3本にするか、TP=2 + 1枚を別用途に回すかのいずれかを選ぶ。

2枚を割り当てる場合の設計判断

構成	得られるもの	向いている状況
TP=2（1モデルを2枚に分割）	メモリ2倍、レイテンシ低下	大きいモデル、応答速度重視
レプリカ2本（TP=1 × 2）	スループット2倍	モデルが1枚に載る、同時実行数重視

配分変更のコスト

2/3/2/1 から 3/2/2/1 へ変更する場合、必要な作業はresource指定の書き換えとローリング再起動のみである。ノードを停止する必要はない。

7.2 サービス数 > GPU数の場合

ここで初めて「GPU共有」の技術が必要になる。

手段	やり方	適する状況
Multi-LoRA	同じベースモデルの派生をアダプタで切替	最も効果大きい。アダプタはMB単位で済む
sleep mode	使っていないサービスをGPUから退避	時間帯で利用に偏りがある場合
メモリ分割	1枚に複数のvLLMを同居	同一信頼境界かつ小さいモデル
MPS	クライアントごとにメモリ上限を設定	信頼できる並行処理

sleep mode とは：サーバを停止せずに、モデル重みとKVキャッシュを含むGPUメモリの大半を一時解放する機能である。Level 1は重みをCPU側のRAMへ退避し、Level 2は破棄する。どちらもフルリロードより18～200倍速く復帰し、TP / PP / EPと併用できる。

SLEEP MODE の既知の不具合

マルチモーダル系の特定オプション使用時に、スリープが成功を返してもHBMが解放されないという不具合報告がある。導入前に実機で検証すべきである。

7.3 マルチノード構成

「マルチノード」という語は2つの別物を指すため、分けて整理する。

意味	中身	頻度
① 1インスタンスが複数ノードにまたがる	TP / PPでモデルを分割	巨大モデル専用。稀である
② 複数ノードでサービスを運用	各ノードに独立したレプリカ	ごく普通である

②はノード間でGPU通信が発生せず、障害耐性の問題もない。スループットを上げたいだけなら常にこちらを選ぶ。

単一ノードで足りる境界線

H200 8枚で 1,128GB が1ノードの器の大きさとなる。

モデル	精度	重み	1ノードに収まるか
70B	BF16	約 140 GB	余裕 (1枚でも可)

モデル	精度	重み	1ノードに収まるか
405B	BF16	約 810 GB	収まる
DeepSeek 671B	FP8	約 670 GB	収まる
Kimi K2 (1T)	FP8	約 1,000 GB	ギリギリ
Kimi K3 (2.8T)	FP8	約 2,800 GB	入らない

おおよそ1兆パラメータ（FP8）が境界線である。これを超えなければ単一ノードで完結する。

2ノード構成の組み方



```

# サーバ1 (rank 0) – APIを持つ
vllm serve /path/to/model \
  --tensor-parallel-size 8 --pipeline-parallel-size 2 \
  --nnodes 2 --node-rank 0 --master-addr <サーバ1のIP>

# サーバ2 (rank 1) – headless
vllm serve /path/to/model \
  --tensor-parallel-size 8 --pipeline-parallel-size 2 \
  --nnodes 2 --node-rank 1 --master-addr <サーバ1のIP> --headless
  
```

効率的なテンソル並列にはInfiniBandのような高速ネットワークアダプタによるノード間通信が必要である。クラスタでInfiniBandを使う場合は `--privileged -e NCCL_IB_HCA=mlx5` のような引数を追加する。GPUDirect RDMAを使うと、ネットワークアダプタがCPUとシステムメモリを経由せずGPUメモリへ直接アクセスできる。

組み方は2通りある。 `--nnodes / --node-rank` を使うネイティブのheadlessモードはRayを必要とせず簡潔である。従来からのRayクラスタ方式も利用できる。Kubernetesで組む場合は **LeaderWorkerSet (LWS)** が対応するAPIとなる。「1インスタンス = 複数Podのまとまり」を表現できる仕組みで、通常のDeploymentではこれを表現できないためである。

障害耐性がない

どちらかのノードが落ちるとインスタンス全体が停止する。**マルチノードは「1台に収まらないモデル」のための手段であり、可用性のための冗長化ではない。**冗長化したい場合は、同じ構成をもう1セット用意してロードバランサで振り分ける。

巨大モデル以外でマルチノードになるケース

- **超長文脈でKVキャッシュが足りない** — モデルは収まるが、100万トークン級の文脈を多数保持できない場合
- **Prefill / Decode分離** — 入力処理用と生成用でノード群を分ける構成。NVIDIA Dynamoやllm-dが採用する
- **MoEのExpert Parallelism** — 専門家をノード間に分散させる方式

Kubernetes か ベアメタルか

Kubernetesは前提条件ではない。本レポートで扱った仕組みは、ほぼすべてがKubernetesと無関係のレイヤにある。

8.1 Kubernetesなしでも使えるもの

- `CUDA_VISIBLE_DEVICES` によるGPU割当
- `--gpu-memory-utilization` によるメモリ分割
- TP / PP / DP
- sleep mode、Multi-LoRA
- **MPS** — デーモンであり、同様にKubernetesは無関係である
- DCGM Exporter + Prometheus による監視

```
# /etc/systemd/system/vllm-a.service
[Service]
Environment="CUDA_VISIBLE_DEVICES=0,1"
ExecStart=/usr/local/bin/vllm serve model-a \
  --tensor-parallel-size 2 --port 8001
Restart=always

[Install]
WantedBy=multi-user.target
```

8.2 決定的な違い — 強制力

方式	GPU割当の性質
systemd + <code>CUDA_VISIBLE_DEVICES</code>	お約束にすぎない。環境変数を付け忘れたプロセスは全GPUを掴む
Docker <code>--gpus '"device=0,1"'</code>	強制される。コンテナから他のGPUは見えない
Kubernetes device plugin	強制 + 台帳管理

実務的な落としどころ

ベアメタルで運用するなら **Docker Compose** を挟むのが現実的である。systemdの手軽さとコンテナの強制力が両立する。

```

services:
  svc-a:
    image: vllm/vllm-openai:latest
    command: --model model-a --tensor-parallel-size 2
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              device_ids: ['0', '1']
              capabilities: [gpu]

```

8.3 Kubernetesを導入する価値が出る条件

条件	Kubernetesの要否
1台・サービス数固定・配分固定	不要。Docker Composeで十分である
ノードが2台以上	検討の価値がある
配分を頻繁に変更する	あったほうが楽である
複数チームが利用する（利用状況の台帳が必要）	必要である
ローリング更新・オートスケールが要件	必要である

1台8枚に4サービスという構成であれば、Kubernetesは明確に過剰である。Docker Compose + nginx + DCGM / Prometheus で必要十分であり、障害時の切り分けも容易になる。

8.4 DRAについて

DRA（Dynamic Resource Allocation）とは： Kubernetesが「GPUを1個ください」ではなく「メモリ80GB以上でこの世代のGPU」のように**条件で要求できる**仕組みである。従来のdevice pluginは整数でしか要求できなかった。

DRAはKubernetes 1.34で正式版となり、1.35で機能ゲートが固定された。最新のv1.36ではさらに機能が拡張され、メモリやCPUといったネイティブリソースへの対応やPodGroupでのResourceClaim対応が加わっている。

DRAへの誤解

DRAは**分割そのものを行う機能ではない**。DRAはデバイスの選択肢と要求を表現できるが、実際の割当や共有はデバイスプラグインやベンダードライバが担う。

ベアメタル構成の実装

第8章でKubernetesが必須でないことを述べた。本章では、実際にKubernetesを使わずに複数サーバへ配分する場合の具体的な組み方を示す。

想定するのは H200を8枚搭載した物理サーバが4台（合計32枚） という構成である。

9.1 構成要素は3つである

要素	役割	変更の頻度と手間
① vLLMインスタンス群	モデルとGPU割当を定義して起動	変更時はコンテナ再起動
② ゲートウェイ	モデル名からバックエンドへの対応表	変更は即時に反映される
③ サービス側の設定	入口URLを1行書くのみ	基本的に変更しない

ゲートウェイとは：複数の推論エンジンの手前に置く中継役である。クライアントからは1つの窓口に見せ、内部で適切なバックエンドへ振り分ける。リバースプロキシ、ルータ、LLMゲートウェイなどとも呼ばれる。

9.2 サービス設定にエンドポイントを直接書いてはならない

最も陥りやすい設計上の誤りが、サービス側の設定ファイルにvLLMのアドレスを直接記載することである。

アンチパターン

サービス側に `http://10.0.0.11:8001` と書いた場合、次のすべてが**全サービスの設定変更と再デプロイ**を伴う。

- GPUの割当を変更したとき
- サーバが1台故障したとき
- レプリカを1本増やしたとき

推論基盤側の都合が、そのままアプリケーション側の再デプロイになる。**4台32枚の規模ではこの運用は破綻する。**

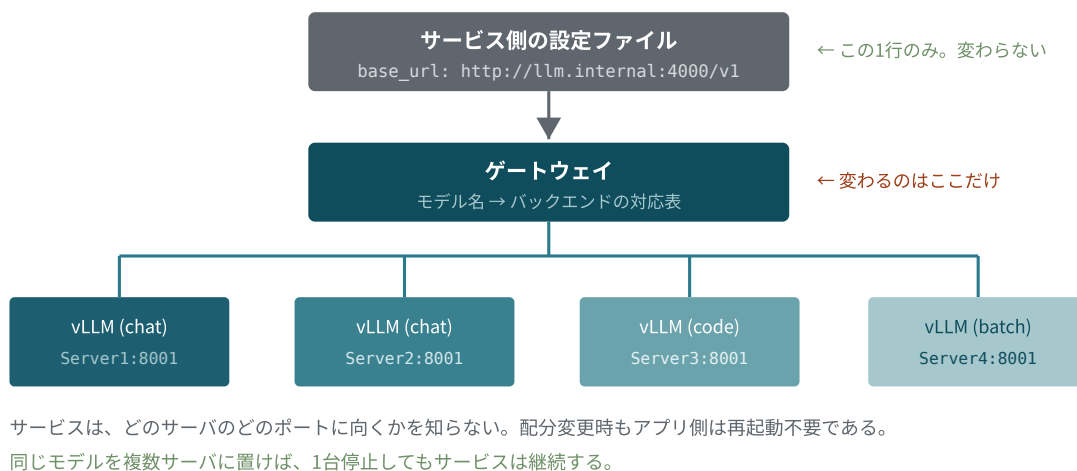


Fig 9.1 – ゲートウェイを挟むことで、サービス側の設定と推論基盤の構成が切り離される

9.3 選び分けはモデル名で行う

vLLMはOpenAI互換のAPIを提供する。このためサービス側は標準のOpenAI SDKをそのまま使い、**URLではなく model パラメータでバックエンドを指定できる。**

OpenAI互換APIとは：OpenAIが定めたリクエスト形式（`/v1/chat/completions` など）に合わせたAPIのことである。これに準拠していれば、クライアント側は接続先が何であることを意識せずに済む。

```
client = OpenAI(base_url="http://llm.internal:4000/v1", api_key="...")

client.chat.completions.create(
    model="chat-70b",      # ここを変えるだけでバックエンドが切り替わる
    messages=[...]
)
```

9.4 ゲートウェイの実装

素のnginxでは、リクエストボディ内の **model** を見て振り分けるのが煩雑である。この用途には **LiteLLM Proxy** のようなLLM向けゲートウェイが広く使われている。

LiteLLM Proxyは、Ollama、vLLM、OpenAI、Anthropic、Googleなどのモデルを単一のOpenAI互換エンドポイントの背後に統合する。リバースプロキシとして動作し、認証、モデルルーティング、ロードバランシング、フォールバック連鎖、レート制限、利用量追跡を1つのYAML設定ファイルから扱う。アプリケーション側は1つのエンドポイントを1つのAPIキー形式で呼ぶだけでよく、モデルの差し替えやプロバイダの追加をコード変更なしで行える。


```

model_list:
- model_name: chat-70b                                # サービスが指定する名前
  litellm_params:
    model: hosted_vllm/Qwen3-70B                      # hosted_vllm/ でvLLMとして扱う
    api_base: http://10.0.0.11:8001                   # Server 1

- model_name: chat-70b                                # 同じ名前 = 自動で負荷分散される
  litellm_params:
    model: hosted_vllm/Qwen3-70B
    api_base: http://10.0.0.12:8001                   # Server 2

- model_name: code-completion
  litellm_params:
    model: hosted_vllm/Qwen3-Coder
    api_base: http://10.0.0.13:8001                   # Server 3

```

同一の `model_name` を複数記述すれば、それだけでレプリカ間の振り分けとなる。

9.5 ゲートウェイのリスク

ゲートウェイは全リクエストと全APIキーを経由する。**侵害された場合の被害範囲が極めて大きい**ため、導入にあたっては次の2点を設計に織り込む必要がある。

単一障害点になる

ゲートウェイが停止すると全サービスが停止する。実運用では2台構成とし、その前段にDNSラウンドロビンまたはVIPを置くのが定石である。

サプライチェーン攻撃の標的になる

実例：LITELLM の PYPI パッケージ侵害（2026年3月）

2026年3月24日、LiteLLMのPyPI公開版のうち **1.82.7** と **1.82.8** に悪意あるコードが混入した状態で公開された。PyPIによる隔離までの約40分間に、数万件のダウンロードが発生している。

1.82.8 では `litellm_init.pth` というファイルが仕込まれていた。Pythonの `.pth` ファイルはインタプリタ起動時に自動実行されるため、`import litellm` を書いていなくてもコードが動作する。認証情報とシステム情報が攻撃者のサーバへ送出された。

侵害の起点はLiteLLM自身ではない。同社の公式アドバイザーは、CI/CDのセキュリティスキャン工程で使用していた **Trivy の侵害に由来する**との見解を示している。上流ツールの侵害が下流パッケージの汚染へ連鎖した事例である。

影響を受けないバージョン：1.82.7 より前、および **v1.83.0 以降**（事案後の最初のクリーンリリース。分離環境と強化されたセキュリティゲートを備えた新CI/CDパイプラインで公開された）。

閉域環境での運用指針

- バージョンを固定し、更新は意図的な作業として実施する
- コンテナイメージのハッシュを検証してから取り込む
- 該当バージョンを導入した環境は、パッケージの有無ではなく認証情報の全面的な露出として扱う
- ゲートウェイからの外向き通信を監視・制限する

9.6 起動時に固定されるもの、実行中に変えられるもの

対象	いつ決まるか	備考
どのGPUを使うか	起動時に固定	コンテナ起動時に確定する
ベースモデル	起動時に固定	—
メモリ配分	起動時に固定	vLLMが先に確保するため
LoRAアダプタ	実行中に変更可	本番利用は非推奨（下記参照）
sleep / wake	実行中に変更可	—
ルーティング先	実行中に変更可	ゲートウェイ側で完結する

LoRAとは：ベースモデルに小さな差分だけを追加して用途別に振る舞いを変える手法である。アダプタ1つあたりの容量はMB単位で済むため、同一ベースモデルの派生を多数扱う場合に効率が良い。

LoRAの動的ロードに関する注意

vLLMは `VLLM_ALLOW_RUNTIME_LORA_UPDATING=True` を設定することで `/v1/load_lora_adapter` と `/v1/unload_lora_adapter` のエンドポイントを有効化でき、実行中にアダプタを追加・削除できる。

ただしvLLM公式はこの機能にセキュリティ上のリスクがあると明記しており、隔離された完全に信頼できる環境でない限り本番利用すべきでないとしている。有効化するとAPIサーバが警告ログを出力する。利用者がモデルのアダプタ管理に介入できてしまうためである。

9.7 柔軟性の実体

ここが本構成の性格を決める部分である。**Kubernetesがない構成にスケジューラは存在しない**。配置を決めるのは人間であり、その決定は設定ファイルに記述されるだけである。

手段	反映までの時間	用途
compose定義の書き換えと再起動	分オーダー	恒久的な配分変更
sleep mode	秒オーダー	時間帯での使い分け

手段	反映までの時間	用途
ゲートウェイの振り分け変更	即時	トラフィックの寄せ・抜き

裏返しの利点

自動で最適配置する仕組みは存在しない。しかし同時に、**意図しない再配置も起きない**。ノードが4台程度であれば、この予測可能性はむしろ運用上の利点となる。

9.8 配置の例

32枚（4台）の配置例

Server 1 チャット TP=2 × 4本	Server 2 チャット×2 / 補完×4	Server 3 補完×2 / 夜間バッチ / 予備	Server 4 検証・実験枠
----------------------------	---------------------------	-------------------------------	--------------------

- チャットを Server 1 と 2 にまたがらせているのが**冗長化**である。1台停止してもサービスは継続する
- Server 4 を丸ごと空けているのが**検証枠**である。第10章の検証はここで実施できる
- Server 3 に予備2枚を残し、負荷増加時の緩衝材としている

9.9 Kubernetesなしで自前になるもの

機能	代替手段	難易度
ルーティング	LiteLLM / nginx / HAProxy	容易
プロセス再起動	<code>restart: always</code>	容易
設定の一元管理	Ansible など	容易
監視	DCGM Exporter + Prometheus	容易（Kubernetes不要で動作する）
モデルファイルの配布	各サーバへ配置、または共有ストレージ	やや手間
ローリング更新	自前スクリプト	手間がかかる
オートスケール	—	実現できない

上の4つは素直に用意できる。**負担となるのは下の3つ**であり、ここがKubernetesを導入するかどうかの分水嶺となる。

検証計画 — 0円から始める

検証したい内容を2層に分解すると、**すべて費用をかけずに確認できる**。



10.1 層1 — GPUなしでKubernetesの割当を検証する

Kubernetes SIG Node が管理する **dra-example-driver** は、物理ハードウェアなしでGPUをシミュレートする。実際の計算は走らないが、スケジューラの割当挙動は本物と同一である。必要なのは Docker Desktop、kind、kubectln、helm、RAM約4GB、そして30分程度である。

kind とは： Kubernetes IN Docker の略。Dockerコンテナの中にKubernetesクラスタを丸ごと立てるツールであり、ノートPCで完結する。

本番移行は、example driverをNVIDIAの実ドライバに差し替え、セレクトアを実際の属性（compute capability、メモリ、トポロジ）へ向けるだけでよい。**8枚のGPUをシミュレートすれば、7.1節の2/3/2/1という割当も含めて検証できる**。

10.2 層2 — 無料枠で使えるGPU

サービス	内容	クレカ
Google Colab / Kaggle	合わせて週60時間超のT4	不要
Lightning AI	月15クレジット（安価GPU換算で約80 GPU時間）、永続クラウドIDE、ストレージ50GB。無料Studioは4時間ごとに手動再起動が必要	不要
Hugging Face ZeroGPU	H200を共有利用。無料枠は1日3.5分、PRO（\$9/月）で1日25分	不要

検証機の選び方

- **T4は避けるべきである。**Turing世代（compute capability 7.5）であり、bfloat16とFP8はcompute capability 8.0以上が必要となる。またFlashAttentionはAmpere（sm80）以上であり、モデルによっては対応バックエンドが1つも存在しない場合がある
- **L4 または L40S を選ぶ。**Ada世代（8.9）でありBF16もFP8も通る
- **ZeroGPUはSpaces専用である。**任意のサーバプロセスやKubernetesは立てられない

なお、attentionバックエンドの既定値は世代で変わる。Blackwell（SM100+）ではFA4、Hopper（SM90）ではFA3、それ以外はFA2が既定となる。バックエンドが変わればワークスペースのメモリ確保量も変わるため、これも数値が環境をまたいで移らない一因である。

10.3 誤解しやすい「無料」

名称	実態
NVIDIA Build	開発者プログラム登録で1,000クレジット、申請等で最大4,000追加。 APIの無料枠であり、GPUは借りられない
NVIDIA LaunchPad	ハンズオンラボへの無料・一時的アクセス。ラボのシナリオに沿った利用が前提で、任意のクラスタは構築できない
GCP \$300 / Azure \$200	クレカ必須、30～90日で失効。 トライアル自体がGPUアクセスをブロックする 。GPU実行前に有料アカウントへの変換が必要で、新規アカウントはGPUクォータを拒否されることが多い

10.4 推奨する進め方

全工程が無償で完結する

1. **層1を先に固める** — kind + dra-example-driver で割当のYAMLとスケジューリング挙動を確認する。GPU不要、所要30分程度である
2. **層2でエンジン挙動を確認する** — L4 または L40S 1枚で、メモリ分割・sleep mode・Multi-LoRA・TP=3が弾かれる挙動を実測する
3. **本番機で数値を取り直す** — `--gpu-memory-utilization` の値は環境をまたいで移らない（5.1節）。手順のみを持ち込み、値は本番で再測定する

MIGを検討対象から外したことにより、ベアメタルの時間貸しが不要となり、**検証費用は0円**となる。

調達指針

11.1 価格の目安（2026年時点）

対象	価格	備考
8基構成 HGX H200 サーバ	\$320,000 ~ \$420,000	典型値は約 \$370,000。GPU 1基あたり約 \$46,000
DGX H200（8基）	約 \$400,000 ~ \$500,000	OEMとの差は \$30,000~\$130,000（10~30%）
H200 SXM5 カード単体	\$32,000 ~ \$40,000	PCIe版は \$28,000 ~ \$34,000
DGX B300（8基）	約 \$300,000 ~ \$350,000	情報源により幅がある。要見積もり

11.2 価格差の中身

DGXの上乗せ分はハードウェアではなく、NVIDIAのソフトウェアスタックとサポートである。DGXはOEMサーバと同じHGXベースボードを使うが、CPU・メモリ・ネットワーク・ストレージ・ソフトウェア・サポートが固定セットで付く。

OEM間にも差がある。**Supermicroが一貫して最安**であり、DellとHPEはサポート階層に応じた上乗せがある。100台規模の発注は単発見積もりより8~12%安く、**EU / APAC地域は米国比で5~10%高い**。日本での調達ではこの地域差が必ず乗る。

11.3 見落としやすい点

- GPU以外が高い。非GPU部品がカード代に \$75,000~\$90,000 上乗せされる。GPU価格だけで償却する原価モデルは、実際の保有コストを約20%過小評価する
- 納期が長い。物理ハードウェアの調達は30~50週のリードタイムがかかる
- SXMかNVLかを必ず明記する。3.3節参照

ケーススタディ — Kimi K3

前提

Kimi K3の重みは本レポート作成時点（2026年7月23日）で**未公開**である。フルチェックポイントの公開は7月27日予定であり、それまではKimiのアプリとAPI経由でのみ利用可能で、セルフホストはできない。技術レポートも未公開のため、以下は**パラメータ数からの概算**である。

12.1 モデルの仕様

項目	内容
公開日	2026年7月16日
パラメータ数	2.8兆 (MoE)
アーキテクチャ	896個のエクスパートのうち16個をトークンごとに活性化する。Kimi Delta Attention (KDA) と Attention Residuals を採用
文脈長	100万トークン
入力	テキスト・画像・動画。推論は常時オン
API価格	入力 \$3 / 出力 \$15 (100万トークンあたり)

MoEに関する誤解

MoEは計算量を減らすが、**メモリは減らない**。896個のエクスパート全部をVRAMに常駐させる必要がある。「16個しか使わないから軽い」わけではない。

12.2 必要なH200枚数（概算）

精度	重みのみ	実用構成 (KVキャッシュ込み)
BF16 (2バイト)	約 5,600 GB	6ノード48枚以上
FP8 (1バイト)	約 2,800 GB	4ノード32枚
FP4 (0.5バイト)	約 1,400 GB	H200では選べない

FP8 の場合の計算根拠

重み 2,800 GB

1枚の実効容量 約 125 GB (141GB × 0.9 - オーバーヘッド)

3ノード24枚 = 3,000 GB → 重み 2,800 + KV 200 GB ← 動作はするが窒息する

4ノード32枚 = 4,000 GB → 重み 2,800 + KV 1,200 GB ← 実用的である

12.3 H200とB200の比較

H200はHopper世代であり**FP4に対応していない**。FP4はBlackwell（B200 / B300）からである。これがノード数に直接響く。

H200（FP8）－ 4ノード / 32枚

Node 1	Node 2	Node 3	Node 4
--------	--------	--------	--------

B200（FP4）－ 2ノード / 16枚

Node 1	Node 2	不要
--------	--------	----

B200なら重みが1,400GBに半減し、1枚あたり容量も180GBに増えるため、**ノード数が半分で済む**。電力・設置スペース・ノード間ネットワーク・運用の複雑さがすべて半減する。**この規模のモデルを視野に入れるなら、H200よりB200が合理的である**。

12.4 費用の妥当性

H200で4ノード構成を組むと、11.1節の価格から**約148万ドル（2億2千万円前後）**の設備投資となる。一方、K3のAPI価格は入力100万トークンあたり\$3、出力\$15である。

推奨する判断

自己ホストが正当化されるのは、**機密データを外部に出せない場合**、あるいは**トークン消費量が桁違いに大きい場合**に限られる。

現実的には、**まず1ノードで動く1兆パラメータ級以下のモデルで運用を固め、K3クラスは必要性が確定してから検討する**のが妥当である。2.8兆パラメータを自己ホストする前提で設備を組むと、使わなかった場合の損失が大きすぎる。

コマンド早見表

```
# GPU間の接続状態を確認 (NV18 等ならNVLink、PIX/SYS ならPCIe)
nvidia-smi topo -m

# Fabric Manager の状態確認 (NVSwitch搭載機で必須)
systemctl status nvidia-fabricmanager

# 特定GPUだけを見せて vLLM を起動
CUDA_VISIBLE_DEVICES=0,1 vllm serve <model> --tensor-parallel-size 2 --port 8001

# GPU数で割り切れない場合はPPを使う
vllm serve <model> --tensor-parallel-size 1 --pipeline-parallel-size 3

# sleep mode を有効にして起動
VLLM_SERVER_DEV_MODE=1 vllm serve <model> --enable-sleep-mode
```

vLLM起動ログで確認すべき2行

```
GPU KV cache size: N tokens
Maximum concurrency for X tokens per request: N.NNx
```

検討から除外した技術 — MIG

本レポートの構成では **MIG** を採用しない。本付録はその判断根拠を記録として残すものである。本文はMIGを使わない前提で記述している。

MIG (Multi-Instance GPU) とは：1枚のGPUをハードウェアレベルで最大7つの独立したGPUに分割する機能である。分割された各領域は専用のメモリと演算資源を持ち、互いに完全に分離される。対応するのはAmpere世代以降（compute capability 8.0以上）のデータセンター向けGPUに限られ、A100・H100・H200・B200などが該当する。

B.1 採用しない理由

MIGは強い分離を提供する一方で、**柔軟な配分という目的に対しては制約として働く**。

制約	内容
固定プロファイルのみ	1g / 2g / 3g / 4g / 7g のみ。「37GB割り当て」のような指定はできない
稼働中に変更できない	リサイズにはインスタンスの破棄が必要である
変更時にノード drain	全CUDAプロセスを停止する必要がある
NVLink P2Pが無効化される	TPが使えなくなる
帯域も等分される	1/7スライスではHBM帯域も約1/7になる
スループットが落ちる	稼働率は上がるが、GPUあたりの性能は下がる

裏付けとなる記述を挙げる。プロファイルの再構成にはインスタンスの破棄が必要であり、稼働中のMIGインスタンスをワークロードごと破棄せずにリサイズすることはできない。このため**MIGは設定ポリシーであり、ライブなスケジューリングのつまみではない**。またMIGプロファイルはCUDAプロセスが開始する前に設定しなければならず、ジョブ実行中の再構成はできない。

TPとの非互換が決定的である

MIGモードを有効にするとGPUのNVLinkは無効化され、P2P能力を失う（無効化すると復帰する）。本構成は第4章のとおりTPを前提とするため、この時点でMIGとは両立しない。

性能面でも影響がある。1g.10gbスライスが得られるHBM帯域は全体の約1/8である。70BモデルがフルH100で毎秒1000トークン出するのに4g.40gbスライスでは200トークンまで落ちる、という例も報告されている。LLMのデコードはメモリ帯域が律速となるため、この影響は直接効いてくる。

B.2 MIGが必要になる条件

MIGが必須となるのは、次の3条件が**同時に**揃った場合のみである。

- 1 1枚の物理GPUを、**信頼レベルの異なる利用者**で共有する必要がある
- 2 **ハードウェアレベルのメモリ分離と障害分離**が、要件として明記されている
- 3 **ハイパーバイザを使わない**（ベアメタル + Kubernetes直載せ）

本構成が該当しない理由

- **サービス数 ≤ GPU枚数**である。1枚を分け合う必要がそもそも生じない
- H200は141GB、B200は180GBあり、モデル1つに1枚を割り当てて容量が足りる
- 「GPU 1枚 = サービス1つ」は**MIGより強い分離**である。物理的に別チップであるため、メモリも障害も完全に分かれる

B.3 除外による帰結

項目	影響
分離の担保	GPU / ノードの境界で取る。MIGより強い分離が得られる
TPの利用	制約なく利用できる
配分変更の手順	ノードdrainが不要になり、Pod入れ替えのみで完結する
HBM帯域	分割による目減りが発生しない
検証コスト	ベアメタルの時間貸しが不要となり、0円で完結する
GPU稼働率	1枚を埋めきれないサービスがある場合、余剰が生じる

最後の1行が唯一のトレードオフである。ただしこれは、MIG以外の手段（Multi-LoRA、sleep mode、プロセス同居）でも緩和できる。詳細は7.2節を参照のこと。

再検討が必要になる場合

将来的に **サービス数がGPU枚数を大きく上回り、かつ信頼境界の異なるテナントを同一GPUに同居させる要件**が生じた場合には、本判断を見直す必要がある。その際も、まずノードを分ける選択肢を先に検討すべきである。

出典一覧

公式ドキュメント

- vLLM 公式ドキュメント — <https://docs.vllm.ai/en/latest/>
- vLLM リリース一覧 — <https://github.com/vllm-project/vllm/releases>
- NVIDIA MIG ユーザガイド（対応GPU一覧・プロファイル） — <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/>
- NVIDIA Fabric Manager ユーザガイド — <https://docs.nvidia.com/datacenter/tesla/fabric-manager-user-guide/>
- NVIDIA MPS ドキュメント — <https://docs.nvidia.com/deploy/mps/>
- Kubernetes v1.34: DRA が GA に到達 — <https://kubernetes.io/blog/2025/09/01/kubernetes-v1-34-dra-updates/>
- Kubernetes v1.36: DRA の更新 — <https://kubernetes.io/blog/2026/05/07/kubernetes-v1-36-dra-136-updates/>
- LiteLLM ドキュメント（vLLM連携） — <https://docs.litellm.ai/docs/providers/vllm>
- LiteLLM セキュリティアドバイザリ（2026年3月） — <https://docs.litellm.ai/blog/security-update-march-2026>
- Datadog Security Labs：LiteLLM / Telnyx の PyPI 侵害分析 — <https://securitylabs.datadoghq.com/articles/litellm-compromise-d-pypi-teampcp-supply-chain-campaign/>
- vLLM LoRAアダプタ ドキュメント — <https://docs.vllm.ai/en/latest/features/lora/>
- Kimi K3 技術ブログ — <https://www.kimi.com/blog/kimi-k3>
- Kimi API Platform（K3 クイックスタート） — <https://platform.kimi.ai/docs/guide/kimi-k3-quickstart>

価格・市場情報

- H200 サーバ価格（2026） — <https://www.mercatus-ai.com/blog/h200-server-price>
- H200 価格ガイド（2026） — <https://www.mercatus-ai.com/blog/h200-price>
- NVIDIA GPU 価格ガイド — <https://intuitionlabs.ai/articles/nvidia-ai-gpu-pricing-guide>

利用上の注意

価格・製品仕様・ソフトウェアのバージョンは変動が速い領域である。特に第11章の価格と第12章のKimi K3に関する記述は、実際の意思決定にあたってベンダーからの見積もりと公式ドキュメントで再確認することを前提としている。

GPUリソース配分アーキテクチャ検討レポート / 第3版 / 2026年7月23日

対象バージョン：vLLM v0.25.1 ・ Kubernetes v1.36